Data & A.I. 5 / Artificial Intelligence: Machine Learning Lifecycle Project

Alexander Michielsen, Jan Van Sas, Geert De Paepe

Overview

You will work in teams of three to develop and evaluate a supervised Machine Learning solution, starting from a real-world dataset and ending with a final evaluated model. The project may be a classification or regression problem.
The goal is **not** to obtain the highest possible model score or to use the most sophisticated techniques. The focus is on understanding the problem, making methodologically sound choices, using evidence to support those choices, and being able to explain the complete development process.
You will use the provided project notebook template. The **Executive Summary** should present the main reasoning, decisions, results, and conclusions of the project. The remainder of the notebook provides supporting evidence through code, outputs, tables, and visualizations.
During grading, the Executive Summary is the main starting point. The supporting evidence will only be consulted whenever necessary and when it is clearly linked from the summary. Evidence that is not linked from the summary **will not** be considered during grading.

Project Workflow

The project follows five main stages.

1. Problem & Evaluation Design

- Start from the dataset provided to you by the instructor.
- Identify two alternative plausible target variables among the available variables in the dataset and briefly describe the corresponding prediction problems.
- For each alternative, explain why the target could be meaningful and what the resulting prediction problem would represent.
- Continue the remainder of the project using the target variable assigned by the instructor.
- Frame the selected ML problem using the project template.
- Come up with a simple rule-based baseline model that can be implemented without any ML. This baseline should be included in your comparisons.
- Select appropriate evaluation metric(s) and justify them in the context of the problem.

2. Data Understanding, EDA & Preprocessing

- Create a data dictionary for all original variables, including their meaning, semantic type, and other relevant information.
- Perform purposeful EDA (initial and more detailed) to understand the data and identify potential issues. Use visualizations and summary statistics to support your findings.
- Choose a justified train/test split strategy.

- Make sure no data leakage occurs during EDA and preprocessing.
- Connect important findings to decisions using the logic: **Finding** → **Meaning/Diagnosis** → **Action/Consequence**.
- Implement the final preprocessing in a reproducible, leakage-safe pipeline. If you realise later that multiple pipelines are needed for different models, justify this decision and include a clear overview of the different pipelines here as well.

3. Model Development

- **Validation setup**
  - Choose an appropriate validation/cross-validation strategy.
  - Use the same validation strategy and evaluation metric(s) throughout model comparison and hyperparameter tuning.
- **Initial model comparison**
  - Compare 4–5 suitable ML models.
  - In addition, include:
    - The simple rule-based baseline model
    - A suitable Dummy model
  - The ML models should represent at least two different modelling approaches/families (for example linear, distance-based, tree-based, kernel-based). At least one of the models must be an ensemble model.
  - Do not perform hyperparameter tuning at this stage. Use default or reasonable initial settings.
  - Use appropriate preprocessing for each model and include the preprocessing and model together in a reproducible pipeline (see Stage 2).
- **Hyperparameter tuning**
  - Perform hyperparameter tuning for each of the 4–5 ML models using the same validation strategy.
  - For each model, select hyperparameters (at least 2 per model if possible) (using the course material and official scikit-learn documentation). Aim for approximately 15–20 configurations per model.
  - A configuration is one unique combination of hyperparameter values. Individual cross-validation folds do not count as separate configurations.
  - Use an appropriate search approach for the defined search space, such as grid search or randomized search.
  - A smaller search or fewer hyperparameters is acceptable when the meaningful search space is limited or when the model is substantially more computationally expensive. Mention the reasoning in the Executive Summary as well.
  - More configurations do not earn more marks. The goal is to perform a reasonable search, not to exhaustively search all possible hyperparameter values.
- **Model comparison and selection**
  - Keep a concise experiment overview summarising the relevant initial and tuned experiments and their outcomes.
  - Compare the tuned ML models using the same validation procedure. Keep the rule-based and Dummy baselines visible as reference points.
  - Where relevant, discuss how hyperparameter tuning affected the different models compared with their initial performance.
  - Based on the experimental evidence, make a justified choice of the leading candidate model to take forward to the Feature & Model Refinement stage.

4. Feature & Model Refinement

Starting from the leading candidate model identified during model development, investigate whether changes to the feature representation can improve or simplify the solution.
Note: Normally, this can happen as part of model development, but for the purpose of this project, we will address it in a more structured way.
These investigations do not imply that either feature engineering or feature selection is necessary in a real ML solution. The final system should retain them only when supported by evidence.

- Create at least one plausible new feature from the original variables and briefly explain the reasoning behind it.
- Using the same validation procedure, compare the leading candidate model with and without the newly created feature. Based on the evidence, decide whether the feature should be retained. A created feature does not need to improve performance or be included in the final solution.
- Afterwards, add a feature-selection step to the pipeline of the leading candidate model using at least one method seen in class. Make a copy of your pipeline for this purpose.
- Treat the amount of feature selection (for example, a selection threshold or number of selected features) as a hyperparameter. Evaluate at least three different settings and tune this parameter.
- You may keep the earlier found model-hyperparameters from Stage 3 fixed.
- Compare the resulting configurations using the same validation procedure. Also compare against retaining all features.
- Based on the experimental evidence, select and justify the final feature set and model configuration. It is acceptable to retain all features if feature selection does not provide a meaningful benefit.

5. Final Evaluation & Conclusions

- Fit the final pipeline on the complete training set and evaluate it once on the untouched test set using appropriate evaluation metric(s) and appropriate visualizations.
- Also compare test performance against the simple rule-based baseline model and the Dummy model. Discuss whether the final ML solution provides a meaningful improvement over these baselines.
- Perform error analysis and model interpretation (see later in lectures for more info on this)
- Interpret and discuss the results (no result without interpretation). Also, generally state what the model is good at and what its limitations are.

If you decide to include anything additional to the above five stages, make sure it is clearly stated and justified in the Executive Summary.

Notebook Expectations

Use the provided notebook template as the main project deliverable.

- The Executive Summary should contain the important reasoning, decisions, results, and conclusions needed to understand the project.
- Detailed code and output should appear in the corresponding evidence sections and be linked from the Executive Summary where requested. These sections support the claims made in the summary.
- Do not rely on important reasoning being discovered somewhere in the code or detailed output. Important decisions and conclusions should be visible in the Executive Summary.
- The notebook must run from top to bottom without errors, with the relevant output visible. The submitted notebook must be in executed state, with all outputs visible.

- Keep the final notebook focused: remove obsolete experiments and unnecessary output that no longer contributes to the final reasoning.
- Results should be reproducible where appropriate, for example by controlling randomness.

AI-Assisted Work and Code

AI-assisted coding and other support tools are allowed. You are not assessed on your ability to write Python syntax from memory.
However, you remain responsible for everything you submit. Code should be correct, reproducible, appropriately structured, consistent with the methodology you describe, and free from important methodological problems such as data leakage or invalid evaluation.
Sophisticated code does not compensate for weak reasoning, and code that you cannot explain does not demonstrate project ownership.

Assessment

The project consists of a **team submission** and an **individual oral defence**.
The submitted notebook documents the team's Machine Learning Lifecycle work. During the individual oral defence, each student must be able to explain and justify the submitted work, interpret important results, and reason about related questions or alternative situations.
Students are responsible for understanding the **complete submitted project**, including work developed collaboratively or with AI-assisted tools.
A student who cannot demonstrate sufficient understanding and ownership of the submitted project during the individual oral defence may receive a failing overall project grade, regardless of the team notebook score.
The detailed assessment rubric, grading weights, and evaluation procedure will be published separately before the final submission deadline.

Submission

- Submit the completed project notebook and any required supporting files through Canvas.
- Ensure that the submitted notebook contains all relevant outputs and can be executed in order.
- Use clear filenames and include the data or access instructions where required.
- The exact deadline and submission instructions are published on Canvas.
- A late submission will lead to a 0/20 score for the project.

Good luck!